La Forge

Les fragments d'énergie

Le catalyseur fonctionne, mais Lambda reçoit maintenant un nombre variable de fragments d'énergie.

Selon les expériences, il peut en recevoir deux, trois, cinq… parfois beaucoup plus.

> « Une formule rigide casse dès que la matière change. Apprends à accepter ce qui arrive. »

Cette mission est centrée sur les **paramètres rest** et la réutilisation de fonctions.

Vous allez utiliser :

- paramètres rest `...` ;
- `return` ;
- variables locales ;
- appels de fonctions.

Instructions

Complétez les fonctions du fichier `fragments.js`.

1. Additionner des fragments

Implémentez :

```
sumFragments(...fragments)
```

La fonction accepte un nombre variable de valeurs et retourne leur somme.

```
sumFragments(10, 20)
// => 30

sumFragments(5, 10, 15, 20)
// => 50
```

Utilisez une boucle `for...of` pour parcourir `fragments`.

2. Compter les fragments

Implémentez :

```
countFragments(...fragments)
```

```
countFragments(10, 20, 30)
// => 3
```

3. Calculer une moyenne

Implémentez :

```
averageFragments(...fragments)
```

La fonction doit réutiliser `sumFragments(...)`.

Vous pouvez utiliser `fragments.length` pour connaître le nombre de valeurs reçues.

```
averageFragments(10, 20, 30)
// => 20
```

4. Ajouter une charge de base

Implémentez :

```
totalWithBase(base, ...fragments)
```

Le premier paramètre est obligatoire. Tous les suivants sont regroupés dans `fragments`.

```
totalWithBase(100, 10, 20, 30)
// => 160
```

Réutilisez `sumFragments(...)`.

5. Créer une formule lisible

Implémentez :

```
formulaLabel(name, ...values)
```

La fonction retourne une chaîne au format :

```
Nom : N valeurs
```

Exemple :

```
formulaLabel('Fusion', 10, 20, 30)
// => "Fusion : 3 valeurs"
```

6. Vérifier la portée

Implémentez :

```
prepareMixture(name, ...values)
```

Dans la fonction :

1. créez une variable locale `total` avec `sumFragments(...values)` ;
2. retournez :

```
Nom = total
```

Exemple :

```
prepareMixture('Nova', 10, 20, 5)
// => "Nova = 35"
```

La variable `total` doit rester locale à la fonction.

À rechercher si nécessaire

- `JavaScript rest parameters`
- `JavaScript spread function call`
- `JavaScript function scope`
- `JavaScript for of`

▶ Aides